

Lab 2: RGB LED Control Using PWM

CSE 541: Robotics

Gaven Flores

September 12 2026

A. Lab

In this lab I used an Arduino Uno R4 WiFi to control an RGB LED with PWM (Pulse Width Modulation). The goal was to display seven colors. Orange, yellow green, blue, indigo and purple. Each for one second by mixing different brightness levels on the red, green and blue channels of the LED.

PWM is how the Arduino fakes an analog output from a pin. The pin doesn't actually output a voltage. It just switches between 5V and 0V really fast. The percentage of time the pin stays HIGH is called the duty cycle. On the Arduino `analogWrite(pin, value)` controls this, where value goes from 0 (LOW 0% duty cycle) to 255 (always HIGH 100% duty cycle). The switching happens at around 490–980 Hz, which's way too fast for your eyes to notice so the LED just looks like its at a steady brightness.

An RGB LED has three LEDs inside one package (green blue) that share one common pin. There are two types: cathode and common anode. With cathode the shared pin goes to ground and you drive each color pin HIGH to turn it on. So `analogWrite(pin, 255)` is full brightness and 0 is off. With anode the shared pin goes to 5V (VCC) and you drive each color pin LOW to turn it on since current flows from VCC through the LED to the lower pin. This flips the logic: `analogWrite(pin, 0)` is brightness and 255 is off. To convert you use: `analogWrite(pin, 255 - DesiredBrightness)`.

For this lab I used a common-anode RGB LED breakout module. It has four pins: VCC, R, G and B. It already has resistors built onto the board (R1, R2, R3) so I didn't need to add any external ones. I wired it up as: VCC → 5V, R → pin 11 G → pin 10 B → pin 9. All of these pins support PWM on the Uno R4 WiFi.

To get colors you run three PWM signals at the same time. One for each color channel. And set different duty cycles on each. For example turning red all the way on and green partially on (with blue off) gives you orange. Red and blue together at brightness gives you purple.

B. Experiment

Experiment Design

I built up the code in stages of jumping straight to the final program:

First I wrote a simple test to make sure the wiring worked. I cycled through red and blue individually. Turning each on with `analogWrite(pin, 0)` for 500 ms and off with `analogWrite(pin 255)` for 500 ms. Both lit up fine which confirmed the wiring and the inverted common-anode logic.

Then I tried color mixing by turning red and blue on at the time. I got a purple. I also experimented with ratios. Like full blue with half red. And could see the color shift toward whichever channel was brighter.

Next I used a for-loop to fade red from off to full brightness and back. I stepped the `analogWrite` value from 255 to 0 (fade in) and back up to 255 (fade out) with a small delay each step. I also wrote a crossfade between red and blue using `i` and `(255 - i)` in the loop so as red got brighter blue got dimmer at the same rate.

Finally I wrote the seven-color program the lab asked for. I figured out the analogWrite values for each color by taking the RGB values and subtracting from 255 (since its common anode). Here are the values I used:

Color	Red Pin (11)	Green Pin (10)	Blue Pin (9)
Red	0	255	255
Orange	0	90	255
Yellow	0	0	255
Green	255	0	255
Blue	255	255	0
Indigo	180	255	125
Purple	127	255	127

Mistakes and Debugging

I ran into issues along the way. On I wrote `pinMode = (RedPin, OUTPUT)` instead of `pinMode(RedPin, OUTPUT)`. I was treating the function like a variable assignment instead of a function call. I also initially tried to write the pin numbers as `~11` because of the `~` symbol on the board. That's the bitwise NOT operator in C++ and gives the wrong value. The `~` on the board just labels which pins support PWM.

Another mistake was not turning LEDs off between colors. My first loop turned each color on. Never turned the previous one off so they all piled up and the LED was just showing everything at once. I fixed it by setting each pin to 255 (off) before switching to the color.

When I first tried to make purple I tried `int MixedPin = RedPin + BluePin` thinking I could somehow combine two pins into one. That just gave me pin 20 (11 + 9) which doesn't exist. The actual way to mix colors is just calling `analogWrite` on both pins

I also had trouble with my for-loop at first. I wrote `for (int i = 0; i >= 255; i--)`. Since `i` starts at 0 the condition `i >= 255` is false right away so the loop never ran. Flipping the start to 255 and counting down to 0 fixed it.

The biggest issue was the LED being dead. When I first tested all three colors, green wouldn't light up. To figure out if it was the pin or the LED I moved the wire over to pin 11 (which I already knew worked for red) and uploaded a sketch that only targeted green. Still nothing. Since pin 11 was fine for red the problem had to be the module's LED itself. This meant I couldn't display orange, yellow or green since all three need the channel. The code still has the values for those colors though.

C.

This lab taught me how PWM works for controlling LED brightness. It's not an analog voltage, just the pin flipping on and off fast enough that your eye averages it out. I also learned the difference between anode and common-cathode RGB LEDs. With anode the logic is inverted: 0 is full on and 255 is off because the LED turns on when the pin is LOW relative to VCC.

Out of the seven colors I was able to display four of them (blue, indigo, purple) correctly. Orange, yellow and green didn't work because of the green LED on my module but the PWM values in the code are correct for all seven. The

debugging process was actually useful. Figuring out that the green LED was dead by swapping the wire to a known-working pin was an exercise in narrowing down hardware problems.

I also got some practice with coding mistakes like forgetting to turn LEDs off between colors trying to add pin numbers together to mix them and getting for-loop conditions backwards. These are things but they're easy to mess up when you're new, to Arduino.

D. Attachments

Arduino Code

```
int RedPin = 11;
int GreenPin = 10;
int BluePin = 9;

void setup() {
  pinMode(RedPin, OUTPUT);
  pinMode(BluePin, OUTPUT);
  pinMode(GreenPin, OUTPUT);
}

void loop() {
  // display red, orange, yellow, green, blue, indigo, purple
  // one second delay between each color

  // red
  analogWrite(RedPin, 0);
  analogWrite(BluePin, 255);
  analogWrite(GreenPin, 255);
  delay(1000);
  // orange
  analogWrite(RedPin, 0);
  analogWrite(GreenPin, 90);
  analogWrite(BluePin, 255);
  delay(1000);
  // yellow
  analogWrite(RedPin, 0);
  analogWrite(GreenPin, 0);
  analogWrite(BluePin, 255);
  delay(1000);
  // green
  analogWrite(RedPin, 255);
  analogWrite(GreenPin, 0);
  analogWrite(BluePin, 255);
  delay(1000);
  // blue
```

```
analogWrite(RedPin, 255);  
analogWrite(GreenPin, 255);  
analogWrite(BluePin, 0);  
delay(1000);  
  // indigo  
analogWrite(RedPin, 180);  
analogWrite(GreenPin, 255);  
analogWrite(BluePin, 125);  
delay(1000);  
  // purple  
analogWrite(RedPin, 127);  
analogWrite(GreenPin, 255);  
analogWrite(BluePin, 127);  
delay(1000);  
}
```

Circuit Photo

